



University of Wisconsin-Madison

Apples are great

Brandon Han

2025-11-14

Common (1)

template.cpp17 lines

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define repd(i, a, b) for(int i = (b)-1; i >= (a); --i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef vector<ll> vll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);
    cout << fixed << setprecision(6);
}
```

Dijkstra.h07620d, 21 lines

Description: Computes shortest paths from source vertices to all other vertices in a weighted directed graph. Input array should be starting distances of each node, e.g. input[source] = 0 and input[not source] = inf. Unreachable nodes get dist = inf

Usage: Dijkstra dij(n); dij.addEdge(a, b, w);
vll dist(n, inf); dist[source] = 0;
dij.solve(dist);

```
const ll inf = LLONG_MAX;
struct Dijkstra {
    typedef pair<ll, int> plli;
    vector<vector<plli>> eds;
    Dijkstra(int n) : eds(n) {}
    void addEdge(int a, int b, ll w=1) {
        assert(w >= 0);
        eds[a].push_back({w, b});
    }
    void solve(vll &out) {
        priority_queue<plli> pq;
        rep(i, 0, sz(out)) pq.push({out[i], i});
        fill(all(out), inf);
        while(sz(pq) > 0) {
            auto [dist, curr] = pq.top(); pq.pop();
            if(dist >= out[curr]) continue;
            out[curr] = dist;
            for(auto [d, nxt] : eds[curr]) pq.push({dist + d, nxt});
        }
    }
};
```

FractionArithmetic.h4be113, 29 lines

Description: Operators for fraction arithmetic. Will likely break down after a few operations due to overflow.

```
struct Frac {
    ll p, q;
    Frac(ll numer=0, ll denom=1) : p(numer), q(denom) {
        assert(q != 0);
        if(q < 0) { p = -p; q = -q; }
        ll fac = gcd(abs(p), q);
        p /= fac; q /= fac;
    }
    Frac operator+(Frac b) {
        ll denom = lcm(q, b.q);
        return Frac(
            p * (denom / q) + b.p * (denom / b.q),
```

```
        denom
    );
    }
    Frac operator-(Frac b) { return *this + b.negate(); }
    Frac operator*(Frac b) {
        ll fac_pq = gcd(p, b.q);
        ll fac_qp = gcd(q, b.p);
        return Frac(
            (p / fac_pq) * (b.p / fac_qp),
            (q / fac_qp) * (b.q / fac_pq)
        );
    }
    Frac operator/(Frac b) { return *this * b.invert(); }
    Frac negate() { return Frac(-p, q); }
    Frac invert() { return Frac(q, p); }
    friend ostream &operator<<(ostream &st, Frac a) { return st
        << a.p << "/" << a.q; }
};
```